

bok25

基
礎
班

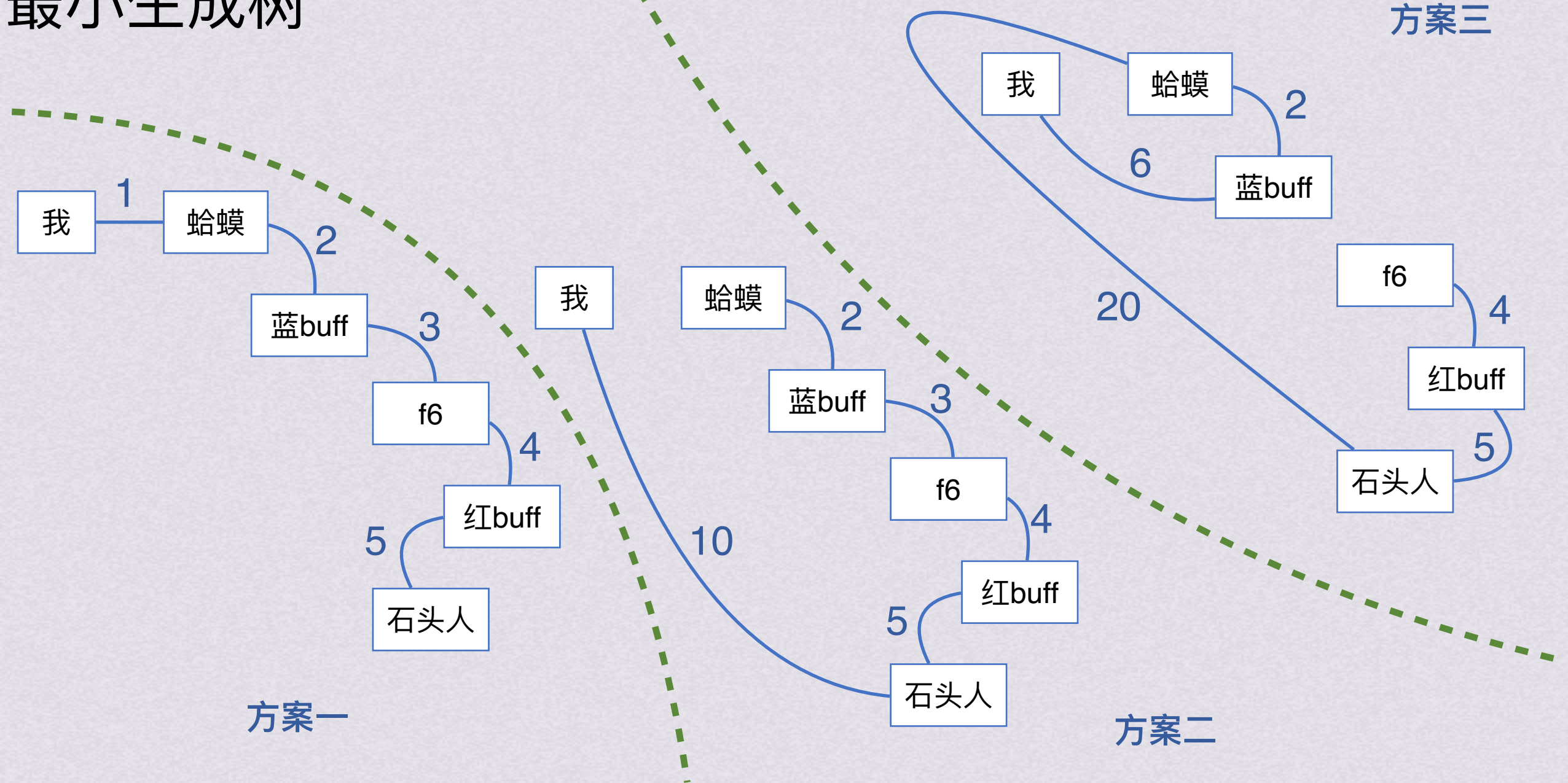
、**考研数据結構**

图的应用

最小生成树

最短路径

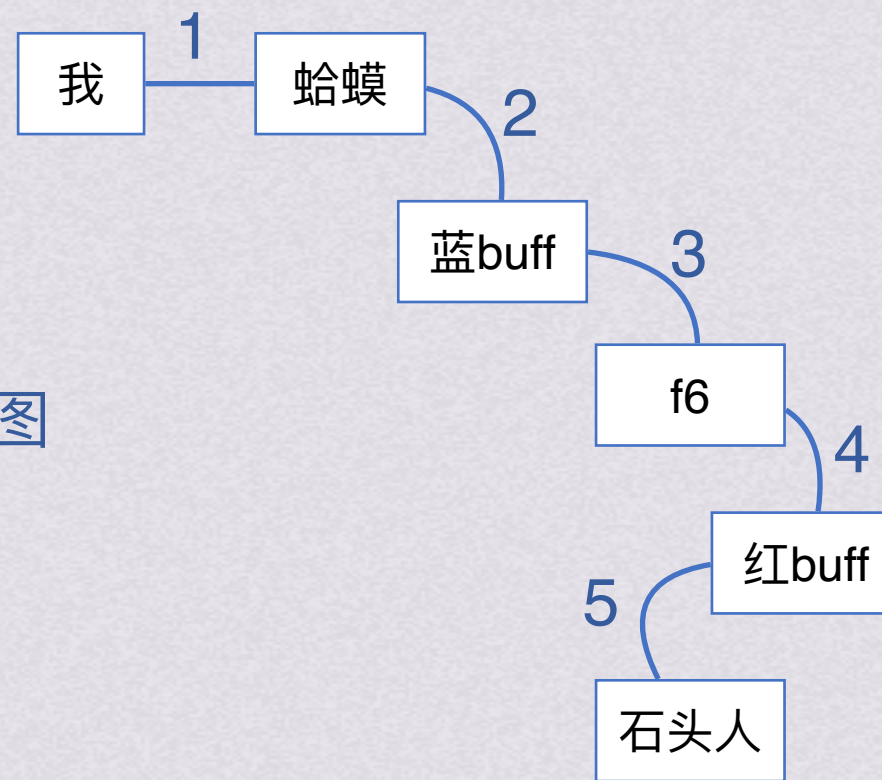
最小生成树





最小生成树

生成树：包含图中所有顶点的一个极小连通子图



方案一

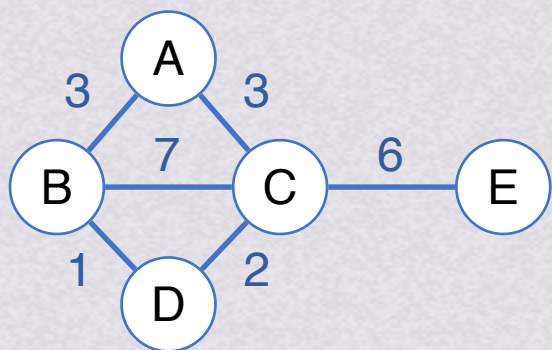


最小生成树

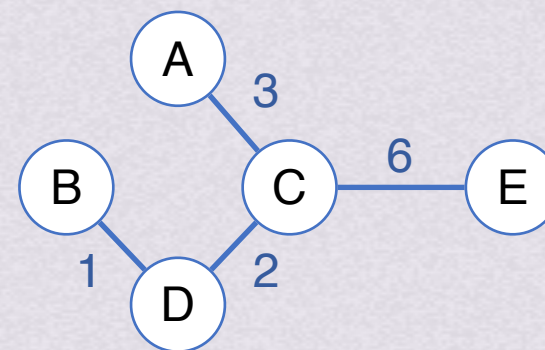
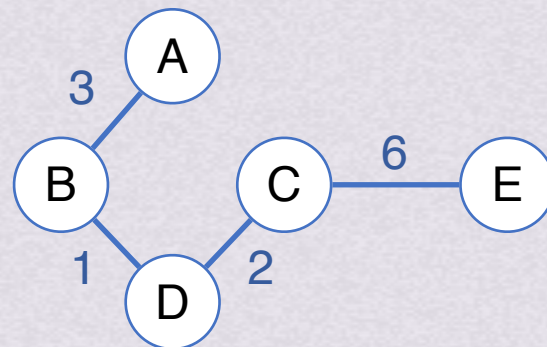
最小生成树：各边代价之和最小的那棵生成树

性质：1. 生成的边的数量一定是 $n-1$

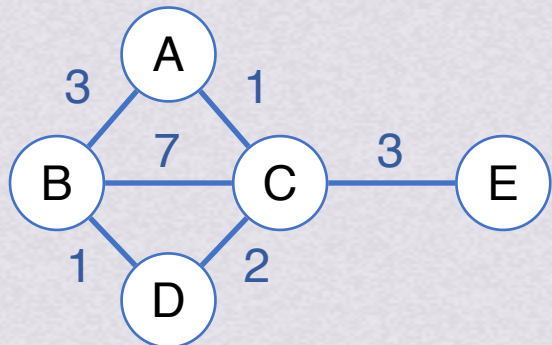
2. 若原图存在权值相同的边，那么该图的最小生成树**可能**不唯一



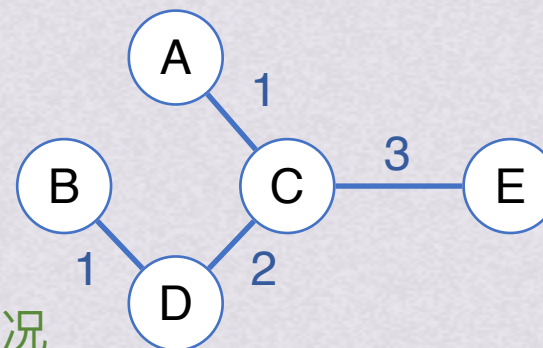
对应最小生成树



不唯一情况



对应最小生成树



唯一情况



最小生成树

最小生成树：各边代价之和最小的那棵生成树

性质：1.生成的边的数量一定是 $n-1$

2.若原图存在权值相同的边，那么该图的最小生成树**可能**不唯一

3.若原图的边权值**互不相等**。那么该图的最小生成树唯一

4.最小生成树中所有边的权值之和是所有生成树中**最小**的



最小生成树

构造最小生成树的两种算法

Prim算法

(适用于边稠密的图)

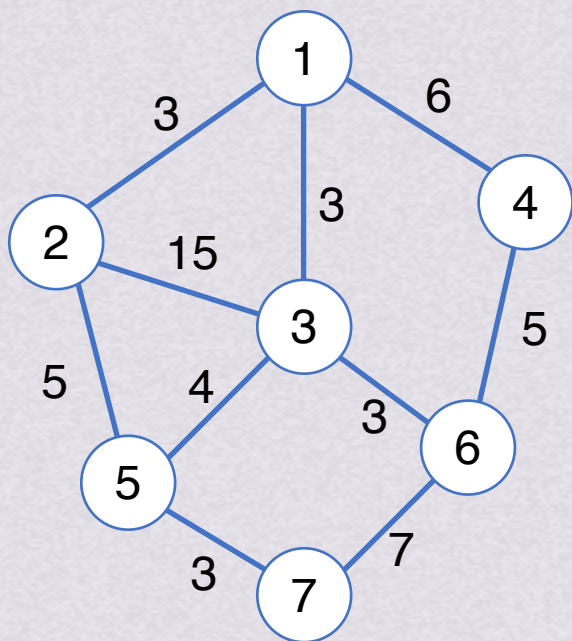
- 步骤：
- 1.初始时任取一顶点加入树T
 - 2.选择一个与当前T中顶点集合距离最近的顶点，并将其顶点和相应边加入T，每次操作后T的顶点数和边数都加一
 - 3.直到所有顶点加入T，此时有 $n-1$ 条边



最小生成树

构造最小生成树的两种算法 **Prim算法** (适用于边稠密的图)

1. 初始时任取一顶点加入树T
2. 选择一个与当前T中顶点集合距离最近的顶点，并将其顶点和相应边加入T，每次操作后T的顶点数和边数都加一
3. 直到所有顶点加入T，此时有 $n-1$ 条边





最小生成树

构造最小生成树的两种算法 **Prim算法** (适用于边稠密的图)

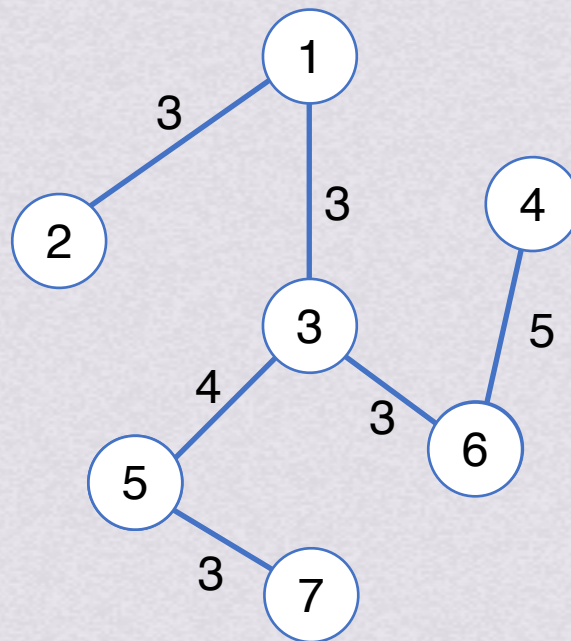
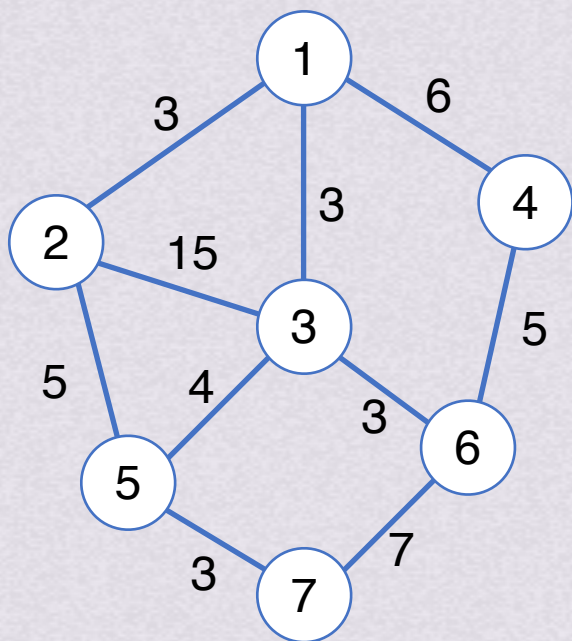
1. 初始时任取一顶点加入树T

2. 选择一个与当前T中顶点集合距离最近的顶点，并将其顶点和相应边加入T，每次操作后T的顶点数和边数都加一

3. 直到所有顶点加入T，此时有n-1条边

时间复杂度: $O(|V|^2)$ 不依赖于E (边数)

MST (最小生成树) 代价: $3+3+3+3+4+5=21$



prim算法生成的最小生成树



最小生成树

构造最小生成树的两种算法

kruskal算法

(适用于边稀疏，顶点多的图)

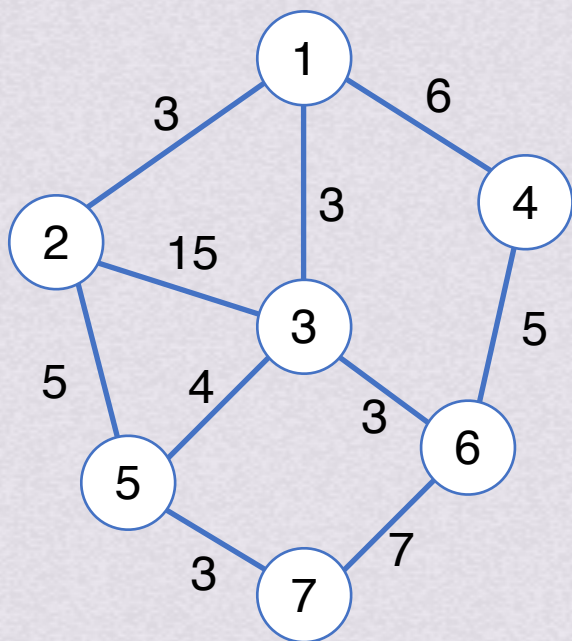
- 步骤：
- 1.初始时为只有 n 个顶点而无边的非连通图 T ，每个顶点自成一个连通分量
 - 2.按照边的权值从小到大的顺序，选择当前未被选取且权值最小的边。若该边依附的顶点落在 T 中不同的连通分量上，则将此边加入 T ，否则舍弃此边而选择下一条
 - 3.直到 T 中所有顶点都在一个连通分量上



最小生成树

构造最小生成树的两种算法 **kruskal算法** (适用于边稀疏, 顶点多的图)

- 1.初始时为只有 n 个顶点而无边的非连通图 T , 每个顶点自成一个连通分量
- 2.按照边的权值从小到大的顺序, 选择当前未被选取且权值最小的边。若该边依附的顶点落在 T 中不同的连通分量上, 则将此边加入 T , 否则舍弃此边二选择下一条
- 3.直到 T 中所有顶点都在一个连通分量上

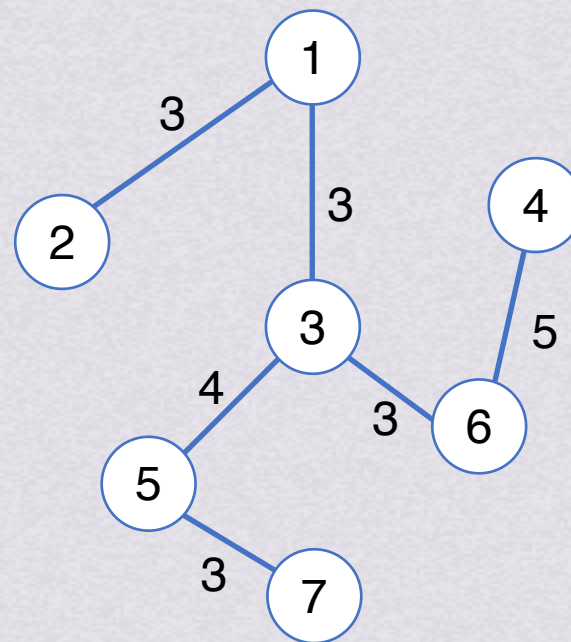
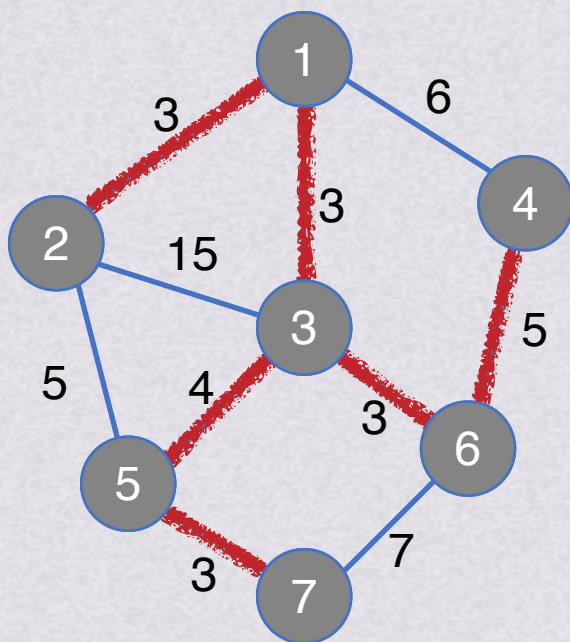




最小生成树

构造最小生成树的两种算法 **kruskal算法** (适用于边稀疏, 顶点多的图)

1. 初始时为只有 n 个顶点而无边的非连通图 T , 每个顶点自成一个连通分量
2. 按照边的权值从小到大的顺序, 选择当前未被选取且权值最小的边。若该边依附的顶点落在 T 中不同的连通分量上, 则将此边加入 T , 否则舍弃此边而选择下一条
3. 直到 T 中所有顶点都在一个连通分量上

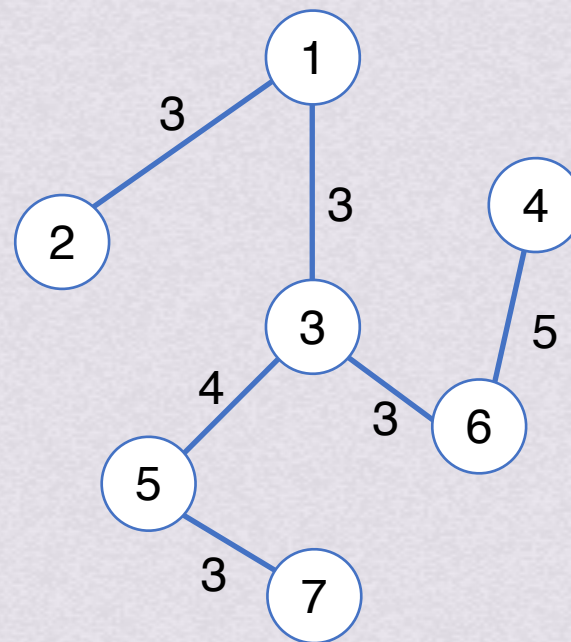
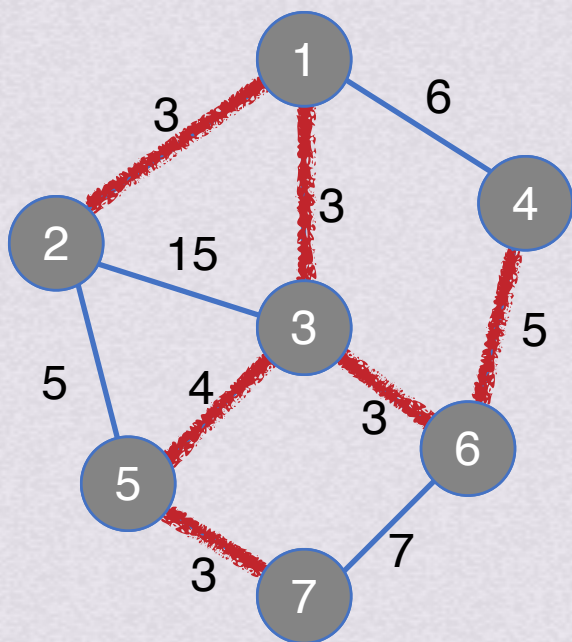




最小生成树

构造最小生成树的两种算法 **kruskal算法** (适用于边稀疏, 顶点多的图)

Kruskal算法的时间复杂度为 $O(|E|\log 2|E|)$, 其中 $|E|$ 是边的个数
由此可知, Kruskal算法复杂度与图中的顶点数无关, 因此适用于稀疏图





最短路径

最短路径： 1.求某个源点到其余各顶点的最短路径（Dijkstra）
2.求每一对顶点之间的最短路径（Floyd）

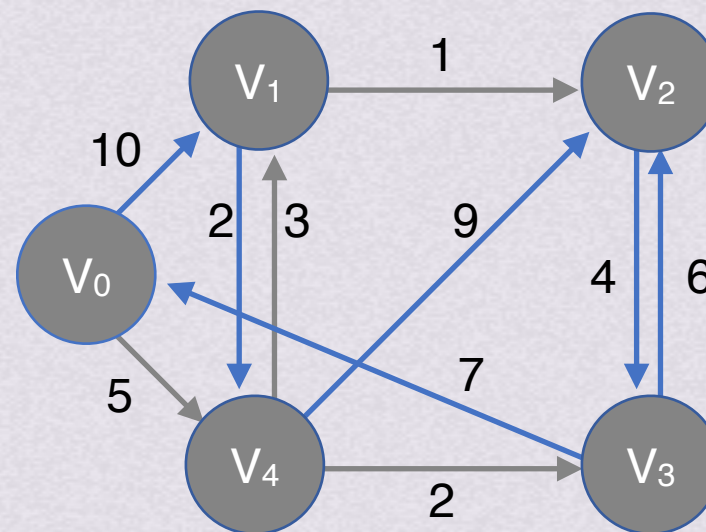


最短路径 Dijkstra (求V0到各点最短路径)

步骤：1.初始化三个数组信息

2.观察final[]数组，若final[i]=0且dist[i]最小，则将final[i]改为1，收录该顶点vi到最优路径集合中

3.对vi的邻接点vj做如下判断：
如果 $\text{dist}[i] + \langle v_i, v_j \rangle$ 的权值 $< \text{dist}[j]$ ，则更新dist[j]与path[j]



标注各顶点是否已找到最短路径 final[5]

1	1	1	1	1
---	---	---	---	---

最短路径长度 dist[5]

0	8	9	7	5
---	---	---	---	---

遍历完毕，该数组为V0到各个顶点的最短距离

路径上的前驱 path[5]

-1	4	1	4	0
----	---	---	---	---

数组下标

0 1 2 3 4

*注意！Dijkstra不适用于边上带有负权值的图！



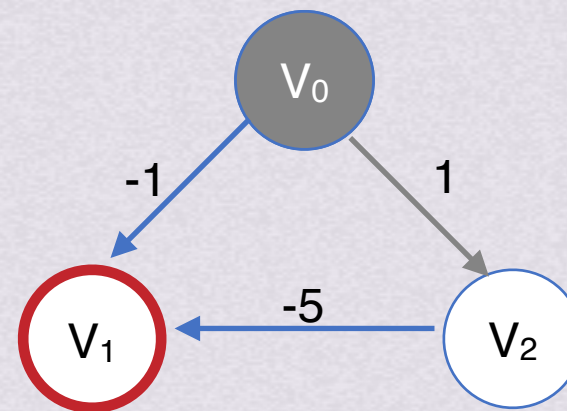
最短路径 Dijkstra (求 V_0 到各点最短路径)

步骤：1.初始化三个数组信息

2.观察final[]数组，若final[i]=0且dist[i]最小，则将final[i]改为1，收录该顶点 v_i 到最优路径集合中

3.对 v_i 的邻接点 v_j 做如下判断：
如果 $\text{dist}[i] + \langle v_i, v_j \rangle$ 的权值 $< \text{dist}[j]$ ，
则更新dist[j]与path[j]

***注意！ Dijkstra不适用于边上带有负权值的图！**



标注各顶点是否已找到最短路径 final[3]

最短路径长度 dist[3]

路径上的前驱 path[3]

数组下标

0 1 2



最短路径BFS方法

(此代码为求解非带权图)

```
bool visited[MVNum]; // 定义一个数组用来记录图中的顶点是否被访问过
void BFS(Graph G, int v) // 广度优先搜索函数, G是图, v是起始顶点
{
    cout << v; // 输出当前顶点
    visited[v] = true; // 标记当前顶点为已访问
    InitQueue(Q); // 初始化一个队列Q
    EnQueue(Q, v); // 将起始顶点v入队

    while (!QueueEmpty(Q)) // 当队列不为空时循环
    {
        DeQueue(Q, u); // 队首元素出队, 并将值赋给u
        // 遍历u的所有邻接顶点
        for (w = FirstAdjVex(G, u); w >= 0; w = NextAdjVex(G, u, w))
        {
            if (!visited[w]) // 如果邻接顶点w未被访问过
            {
                cout << w; // 输出顶点w
                visited[w] = true; // 标记顶点w为已访问
                EnQueue(Q, w); // 将顶点w入队
            }
        }
    }
}
```

```
bool visited[MVNum]; // 定义一个数组用来记录图中的顶点是否被访问过
void BFS_distance(Graph G, int v)
int d[G.vexnum]; // 记录顶点距离的数组
{
    for(i = 0; i < G.vexnum; ++i)
        d[i] = ∞; // 初始化所有顶点的距离为无穷大, 表示一开始所有顶点都不可达

    visited[v] = true; // 标记起始顶点为已访问
    d[v] = 0; // 起始顶点到自己的距离为0
    InitQueue(Q); // 初始化一个队列Q
    EnQueue(Q, v); // 将起始顶点v加入队列

    while (!QueueEmpty(Q)) // 当队列不为空时进行循环
    {
        DeQueue(Q, u); // 队首元素出队, 此元素为当前正在处理的顶点
        // 遍历当前顶点u的所有邻接顶点
        for (w = FirstAdjVex(G, u); w >= 0; w = NextAdjVex(G, u, w))
        {
            if (!visited[w]) // 如果邻接顶点w未被访问过
            {
                visited[w] = true; // 标记顶点w为已访问
                d[w] = d[u] + 1; // 更新顶点w的距离, w的距离是u的距离加上1
                EnQueue(Q, w); // 将顶点w加入队列, 以便后续遍历其邻接点
            }
        }
    }
}
```



最短路径BFS方法

(此代码为求解非带权图)

```
bool visited[MVNum]; // 定义一个数组用来记录图中的顶点是否被访问过
void BFS(Graph G, int v) // 广度优先搜索函数, G是图, v是起始顶点
{
    cout << v; // 输出当前顶点
    visited[v] = true; // 标记当前顶点为已访问
    InitQueue(Q); // 初始化一个队列Q
    EnQueue(Q, v); // 将起始顶点v入队

    while (!QueueEmpty(Q)) // 当队列不为空时循环
    {
        DeQueue(Q, u); // 队首元素出队, 并将值赋给u
        // 遍历u的所有邻接顶点
        for (w = FirstAdjVex(G, u); w >= 0; w = NextAdjVex(G, u, w))
        {
            if (!visited[w]) // 如果邻接顶点w未被访问过
            {
                cout << w; // 输出顶点w
                visited[w] = true; // 标记顶点w为已访问
                EnQueue(Q, w); // 将顶点w入队
            }
        }
    }
}
```

```
bool visited[MVNum]; // 定义一个数组用来记录图中的顶点是否被访问过
void BFS_distance(Graph G, int v)
int d[G.vexnum]; // 记录顶点距离的数组
{
    for(i = 0; i < G.vexnum; ++i)
        d[i] = ∞; // 初始化所有顶点的距离为无穷大, 表示一开始所有顶点都不可达

    visited[v] = true; // 标记起始顶点为已访问
    d[v] = 0; // 起始顶点到自己的距离为0
    InitQueue(Q); // 初始化一个队列Q
    EnQueue(Q, v); // 将起始顶点v加入队列

    while (!QueueEmpty(Q)) // 当队列不为空时进行循环
    {
        DeQueue(Q, u); // 队首元素出队, 此元素为当前正在处理的顶点
        // 遍历当前顶点u的所有邻接顶点
        for (w = FirstAdjVex(G, u); w >= 0; w = NextAdjVex(G, u, w))
        {
            if (!visited[w]) // 如果邻接顶点w未被访问过
            {
                visited[w] = true; // 标记顶点w为已访问
                d[w] = d[u] + 1; // 更新顶点w的距离, w的距离是u的距离加上1
                EnQueue(Q, w); // 将顶点w加入队列, 以便后续遍历其邻接点
            }
        }
    }
}
```

记录从源点开始的遍历过程中访问过顶点的情况



最短路径BFS方法

(此代码为求解非带权图)

```
bool visited[MVNum]; // 定义一个数组用来记录图中的顶点是否被访问过
void BFS(Graph G, int v) // 广度优先搜索函数, G是图, v是起始顶点
{
    cout << v; // 输出当前顶点
    visited[v] = true; // 标记当前顶点为已访问
    InitQueue(Q); // 初始化一个队列Q
    EnQueue(Q, v); // 将起始顶点v入队

    while (!QueueEmpty(Q)) // 当队列不为空时循环
    {
        DeQueue(Q, u); // 队首元素出队, 并将值赋给u
        // 遍历u的所有邻接顶点
        for (w = FirstAdjVex(G, u); w >= 0; w = NextAdjVex(G, u, w))
        {
            if (!visited[w]) // 如果邻接顶点w未被访问过
            {
                cout << w; // 输出顶点w
                visited[w] = true; // 标记顶点w为已访问
                EnQueue(Q, w); // 将顶点w入队
            }
        }
    }
}
```

```
bool visited[MVNum]; // 定义一个数组用来记录图中的顶点是否被访问过
void BFS_distance(Graph G, int v)
int d[G.vexnum]; // 记录顶点距离的数组
{
    for(i = 0; i < G.vexnum; ++i)
        d[i] = ∞; // 初始化所有顶点的距离为无穷大, 表示一开始所有顶点都不可达

    visited[v] = true; // 标记起始顶点为已访问
    d[v] = 0; // 起始顶点到自己的距离为0
    InitQueue(Q); // 初始化一个队列Q
    EnQueue(Q, v); // 将起始顶点v加入队列

    while (!QueueEmpty(Q)) // 当队列不为空时进行循环
    {
        DeQueue(Q, u); // 队首元素出队, 此元素为当前正在处理的顶点
        // 遍历当前顶点u的所有邻接顶点
        for (w = FirstAdjVex(G, u); w >= 0; w = NextAdjVex(G, u, w))
        {
            if (!visited[w]) // 如果邻接顶点w未被访问过
            {
                visited[w] = true; // 标记顶点w为已访问
                d[w] = d[u] + 1; // 更新顶点w的距离, w的距离是u的距离加上1
                EnQueue(Q, w); // 将顶点w加入队列, 以便后续遍历其邻接点
            }
        }
    }
}
```



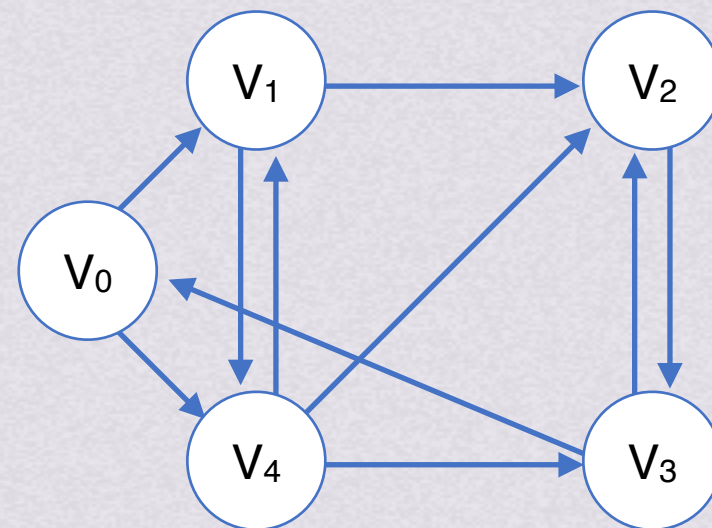
最短路径BFS方法

(此代码为求解非带权图)

```
bool visited[MVNum]; // 定义一个数组用来记录图中的顶点是否被访问过
void BFS_distance(Graph G, int v)
int d[G.vexnum]; // 记录顶点距离的数组
{
    for(i = 0; i < G.vexnum; ++i)
        d[i] = ∞; // 初始化所有顶点的距离为无穷大, 表示一开始所有顶点都不可达

    visited[v] = true; // 标记起始顶点为已访问
    d[v] = 0; // 起始顶点到自己的距离为0
    InitQueue(Q); // 初始化一个队列Q
    EnQueue(Q, v); // 将起始顶点v加入队列

    while (!QueueEmpty(Q)) // 当队列不为空时进行循环
    {
        DeQueue(Q, u); // 队首元素出队, 此元素为当前正在处理的顶点
        // 遍历当前顶点u的所有邻接顶点
        for (w = FirstAdjVex(G, u); w >= 0; w = NextAdjVex(G, u, w))
        {
            if (!visited[w]) // 如果邻接顶点w未被访问过
            {
                visited[w] = true; // 标记顶点w为已访问
                d[w] = d[u] + 1; // 更新顶点w的距离, w的距离是u的距离加上1
                EnQueue(Q, w); // 将顶点w加入队列, 以便后续遍历其邻接点
            }
        }
    }
}
```



图G

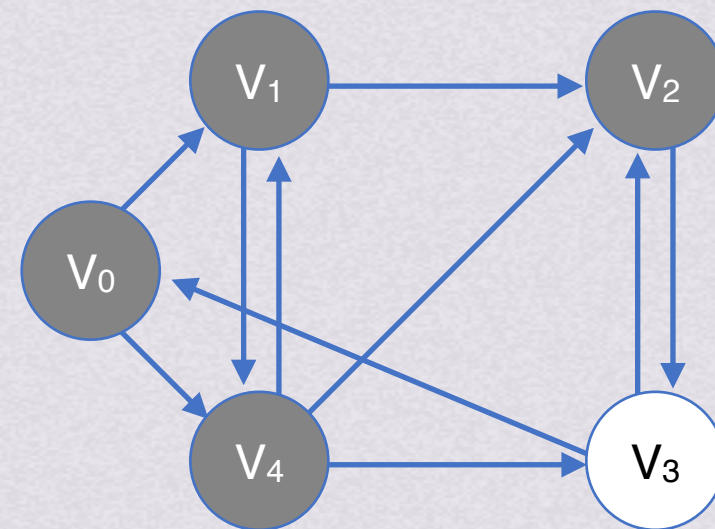
最短路径BFS方法

(此代码为求解非带权图)

```
bool visited[MVNum]; // 定义一个数组用来记录图中的顶点是否被访问过
void BFS_distance(Graph G, int v) 传入G,0
int d[G.vexnum]; //记录顶点距离的数组
{
    for(i = 0; i < G.vexnum; ++i)
        d[i] = ∞; // 初始化所有顶点的距离为无穷大, 表示一开始所有顶点都不可达

    visited[v] = true; // 标记起始顶点为已访问
    d[v] = 0; // 起始顶点到自己的距离为0
    InitQueue(Q); // 初始化一个队列Q
    EnQueue(Q, v); // 将起始顶点v加入队列

    while (!QueueEmpty(Q)) // 当队列不为空时进行循环
    {
        DeQueue(Q, u); // 队首元素出队, 此元素为当前正在处理的顶点
        // 遍历当前顶点u的所有邻接顶点
        for (w = FirstAdjVex(G, u); w >= 0; w = NextAdjVex(G, u, w))
        {
            if (!visited[w]) // 如果邻接顶点w未被访问过
            {
                visited[w] = true; // 标记顶点w为已访问
                d[w] = d[u] + 1; // 更新顶点w的距离, w的距离是u的距离加上1
                EnQueue(Q, w); // 将顶点w加入队列, 以便后续遍历其邻接点
            }
        }
    }
}
```



图G



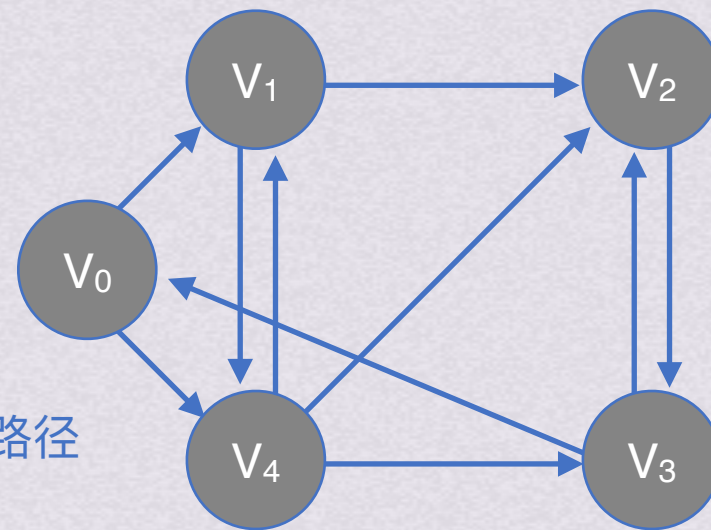
辅助队列Q

d[5]	0	1	2	2	1
visited[5]	1	1	1	1	1

最短路径 BFS方法

d[5]	0	1	2	2	1
------	---	---	---	---	---

数组即为所求非带权图中V0到各顶点最短路径



图G



最短路径

最短路径： 1.求某个源点到其余各顶点的最短路径（Dijkstra）
2.求每一对顶点之间的最短路径（Floyd）



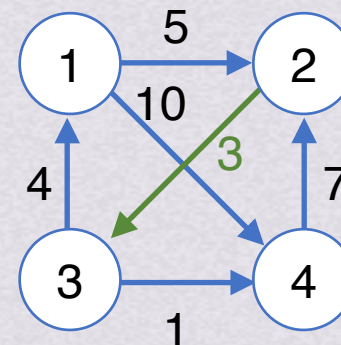
最短路径 Floyd

步骤：1.初始化矩阵

2.选择顶点i作为中转点

	1	2	3	4
1	0	5	∞	10
2	∞	0	3	∞
3	4	∞	0	1
4	∞	7	∞	0

A^{-1}





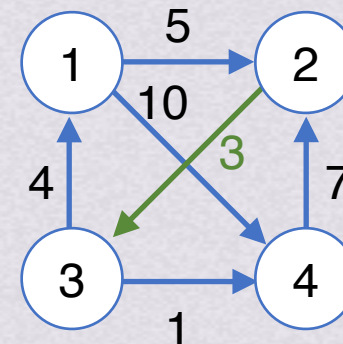
最短路径 Floyd

步骤：1.初始化矩阵

2.选择顶点i作为中转点

	1	2	3	4
1	0	5	8	9
2	7	0	3	4
3	4	8	0	1
4	14	7	10	0

A^4



Floyd算法的时间复杂度为 $O(|V|^3)$

允许有负权值的边，但不允许包含带负权值的边组成的回路



最短路径Floyd

步骤：1.初始化矩阵

2.选择顶点i作为中转点

	1	2	3
1	0	2	∞
2	∞	0	-3
3	1	∞	0

Floyd算法的时间复杂度为 $O(|V|^3)$

允许有负权值的边，但不允许包含带负权值的边组成的回路

